

TP3 — Runners GitLab et Executors (version debutant)

Objectifs

A la fin de ce TP, vous saurez :

1. Ce qu'est un Runner GitLab et a quoi il sert
2. Identifier quel Runner execute vos jobs
3. Utiliser les **tags** pour cibler un Runner specifique
4. Comprendre la difference entre Executor **Shell** et Executor **Docker**
5. Utiliser les **services** GitLab CI (ex: base de donnees pour les tests)
6. Lire et comprendre un pipeline multi-images

Prerequis

- Avoir complete le TP1 (jour 1)
- Un projet GitLab avec un pipeline fonctionnel
- Savoir faire `git add`, `git commit`, `git push`

Etape 1 — Creer le projet sur GitLab

1. Allez sur <https://gitlab.com>
2. Cliquez sur **New project** > **Create blank project**
3. **Project name** : `tp3-runners`
4. **IMPORTANT** : **Decochez** *"Initialize repository with a README"*
5. Cliquez sur **Create project**
6. Gardez l'URL HTTPS du projet de cote

****Rappel du TP1**** : on decoche le README pour eviter le conflit de merge au premier push. Si vous l'oubliez, vous aurez l'erreur ****"Updates were rejected"**.

Etape 2 — Comprendre les Runners (theorie rapide)

Qu'est-ce qu'un Runner ?

Un Runner est un **programme qui execute vos jobs CI/CD**. GitLab lui-meme ne fait rien : il lit votre `.gitlab-ci.yml`, cree les jobs, puis les envoie a un Runner disponible.

```
Vous faites git push
|
v
GitLab lit .gitlab-ci.yml et cree les jobs
|
v
GitLab envoie les jobs au Runner
|
v
Le Runner execute les commandes (npm install, npm test, etc.)
|
v
Le Runner renvoie les resultats a GitLab (vert ou rouge)
```

Les 3 types de Runners

Type	Accessible par	Cas d'usage
Shared	Tous les projets GitLab	C'est ce que vous utilisez sur gitlab.com (gratuit, 400 min/mois)
Group	Projets d'un meme groupe	Equipe/departement qui partage un Runner
Specific	Un seul projet	Besoin particulier (GPU, acces reseau, etc.)

Qu'est-ce qu'un Executor ?

L'executor est la **methode** utilisee par le Runner pour executer vos jobs :

Executor	Comment ca marche	Avantage	Inconvenient
Docker	Chaque job tourne dans un conteneur Docker isole	Propre, reproductible	Un peu plus lent
Shell	Les commandes tournent directement sur la machine	Tres rapide	Pas d'isolation

Sur **gitlab.com**, les Shared Runners utilisent l'executor **Docker**. C'est pour ca que vous mettez `image: node:18-alpine` dans vos jobs : ca dit au Runner quelle image Docker utiliser.

Etape 3 — Observer les Runners de votre projet

1. Allez sur votre projet GitLab
2. Menu de gauche : **Settings** > **CI/CD**
3. Développez la section **Runners**
4. Vous verrez :
 - **Assigned project runners** : runners spécifiques à votre projet (probablement vide)
 - **Shared runners** : runners partagés disponibles (actifs sur gitlab.com)

****Que voir ?**** Vous devriez voir plusieurs Shared Runners avec des noms comme ``green-4.saas-linux-small-amd64.runners-manager.gitlab.com``. Ce sont les machines gratuites de GitLab qui exécutent vos pipelines.

Etape 4 — Préparer le code du projet

Le code est fourni dans le dossier `projet-demo/` du jour 2. Nous allons le copier dans un dossier propre.

Dans Git Bash :

```
# Aller dans le dossier du jour 2
cd ~/Desktop/"CI CD"/jour2

# Créer un dossier de travail propre
mkdir tp3-work
cd tp3-work

# Copier les fichiers du projet-demo
cp -r ../projet-demo/src ./
cp -r ../projet-demo/tests ./
cp -r ../projet-demo/scripts ./
cp ../projet-demo/package.json ./
cp ../projet-demo/.gitignore ./
```

Vérifier que tout est là :

```
ls -la
```

Vous devez voir : `.gitignore`, `package.json`, `scripts/`, `src/`, `tests/`.

Tester en local (optionnel mais recommandé) :

```
npm install
npm test
```

Vous devez voir : **Tests: 17 passed, 17 total.**

Puis supprimez les node_modules :

```
rm -rf node_modules
```

Etape 5 — Créer le `.gitlab-ci.yml`

Copiez le fichier `.gitlab-ci.yml` fourni dans le dossier `tp3-runners/` :

```
cp ~/Desktop/"CI CD"/jour2/tp3-runners/.gitlab-ci.yml ./
```

Puis **ouvrez-le dans VS Code** pour le lire et comprendre chaque section :

```
code .gitlab-ci.yml
```

Contenu du `.gitlab-ci.yml` et explications

Le fichier est organisé en **4 sections** :

Section 1 : Variables et stages

```
variables:
  NODE_VERSION: "18"

stages:
  - install
  - test
  - build
```

****Pourquoi un stage `install` séparé ?****

Dans le TP1, on faisait `npm ci` dans le `before\_script` de chaque job. Ici on dédie un job à l'installation des dépendances, et les autres jobs réutilisent le cache. C'est plus rapide quand on a beaucoup de jobs.

Section 2 : Job d'installation

```
install-deps:
  stage: install
  image: node:${NODE_VERSION}-alpine
  script:
    - npm ci --cache .npm --prefer-offline
```

Pourquoi `npm ci` et pas `npm install` ?

`npm ci` est conçu pour la CI : il installe les versions **exactes** du `package-lock.json` et il est plus rapide.

Section 3 : Jobs de test avec images différentes

```
test-node-18:
  image: node:18-alpine

test-node-20:
  image: node:20-alpine
```

Pourquoi tester sur plusieurs versions de Node ?

Votre code peut fonctionner sur Node 18 mais pas sur Node 20 (ou l'inverse). Tester sur plusieurs versions détecte les incompatibilités. C'est ce qu'on appelle une **matrice de tests**.

Section 4 : Job qui affiche les informations du Runner

```
runner-info:
  script:
    - echo "Runner ID : $CI_RUNNER_ID"
    - echo "Runner Description : $CI_RUNNER_DESCRIPTION"
    - echo "Runner Tags : $CI_RUNNER_TAGS"
```

Pourquoi ce job ?

Pour comprendre quel Runner a exécuté votre job. Les variables `CI\_RUNNER\_\*` sont injectées automatiquement par GitLab. C'est très utile en production pour diagnostiquer les problèmes.

Etape 6 — Pousser vers GitLab et observer

```
# Initialiser Git
git init -b main

# Ajouter tous les fichiers
git add .

# Verifier (pas de node_modules !)
git status

# Commiter
git commit -m "init: TP3 runners et executors"

# Connecter au projet GitLab (REMPLACEZ L'URL)
git remote add origin https://gitlab.com/votre-user/tp3-runners.git

# Pousser
git push -u origin main
```

Etape 7 — Analyser le pipeline

1. Allez sur GitLab > votre projet > **Build** > **Pipelines**
2. Cliquez sur le pipeline en cours
3. Vous devez voir **3 stages** avec **5 jobs** :
 - **install** : `install-deps`
 - **test** : `test-node-18`, `test-node-20`, `runner-info`
 - **build** : `build-app`

Exercice : identifier le Runner

4. Cliquez sur le job `runner-info`
5. Dans les logs, cherchez les lignes qui commencent par :
 - **Runner ID** :
 - **Runner Description** :
 - **Runner Tags** :
6. **Notez ces informations** : elles vous disent quelle machine a execute votre job

Exercice : comparer les versions de Node

7. Cliquez sur le job `test-node-18` et cherchez la ligne `Node.js version`
8. Cliquez sur le job `test-node-20` et cherchez la meme ligne
9. **Les versions sont differentes** car chaque job utilise une image Docker differente

****C'est la puissance de l'executor Docker**** : chaque job a son propre environnement isole. On peut tester sur Node 18 et Node 20 en parallele sans conflit.

Etape 8 — Comprendre les tags (theorie)

Les **tags** permettent de cibler un Runner spécifique pour un job.

```
# Sans tags : GitLab choisit n'importe quel Runner disponible
test:
  script:
    - npm test

# Avec tags : GitLab envoie le job UNIQUEMENT aux Runners
# qui ont le tag "docker" ET le tag "linux"
test:
  tags:
    - docker
    - linux
  script:
    - npm test
```

****Quand utiliser les tags ?****

- Votre projet a besoin d'un GPU → tag `gpu`
- Votre deploy doit tourner sur un serveur spécifique → tag `production`
- Vous avez des Runners Linux et Windows → tags `linux` ou `windows`

****Sur gitlab.com****, les Shared Runners acceptent les jobs ****sans tags****. Vous n'avez pas besoin de spécifier de tags pour un usage basique.

Etape 9 — Comprendre les services GitLab CI (theorie)

Les **services** sont des conteneurs Docker supplémentaires qui tournent **a cote** de votre job. Typiquement, ce sont des bases de données.

```
test-avec-bdd:
  image: node:18-alpine
  services:
    - name: postgres:15-alpine
      alias: db
      variables:
        POSTGRES_DB: test_db
        POSTGRES_USER: test_user
        POSTGRES_PASSWORD: test_pass
  variables:
    DATABASE_URL: "postgresql://test_user:test_pass@db:5432/test_db"
  script:
    - npm test
```

****Comment ca marche ?****

1. GitLab lance un conteneur PostgreSQL (`postgres:15-alpine`)
2. Ce conteneur est accessible depuis votre job via le nom ``db`` (l'alias)
3. Votre application se connecte a ``db:5432`` comme si c'etait un serveur distant
4. Quand le job termine, le conteneur PostgreSQL est ****détruit**** (base de données videe)

****Avantage**** : chaque execution de test a une base de données ****vierge et propre****. Pas de pollution entre les tests.

Recapitulatif du TP3

Concept	Ce que vous avez appris
Runner	Programme qui execute les jobs CI/CD
Executor Docker	Chaque job tourne dans un conteneur Docker isole
Images	<code>image: node:18-alpine</code> choisit l'environnement du job
Matrice de tests	Tester sur plusieurs versions (Node 18, Node 20)
Variables <code>CI_RUNNER_*</code>	Identifient quel Runner a execute un job
Tags	Ciblent un Runner specifique pour un job
Services	Conteneurs supplementaires (BDD, Redis) a cote du job

Pour aller plus loin

- Installez un Runner sur votre machine : `docker run gitlab/gitlab-runner:latest`
- Enregistrez-le sur votre projet : Settings > CI/CD > Runners > New project runner
- Testez un pipeline avec et sans tags pour comprendre le ciblage

Fin du TP3. Passez au TP4 (Cache, Artifacts et Variables) pour optimiser votre pipeline.